



Memoria Transaccional



La **memoria transaccional software** (STM) es un modelo de control de concurrencia que funciona como alternativa al uso de **cerrojos, semáforos o monitores**. Su idea central es copiar el comportamiento de las **transacciones de bases de datos**, pero aplicado a la memoria compartida.

Explicación

En lugar de bloquear recursos (como ocurre con mutex, monitores o semáforos), STM propone:

“Realiza todas tus operaciones de forma optimista y valida al final.”

Reemplaza el “bloquea antes de operar” del modelo clásico por “opera optimistamente y valida al final”.

Durante la transacción:

- Se leen valores de memoria compartida → se guardan localmente.
- Se modifican en privado.
- Al final, se intenta validar:
 - Si *nadie más ha modificado esos valores* desde que los leíste → **commit**.
 - Si hubo conflicto → **abort** (roll-back) y **reintento automático**.

Esto elimina el típico problema de:

- ¿Qué pasa si me olvido un `.unlock()`?
- ¿Qué pasa si dos hilos adquieren recursos cruzados?
- ¿Qué pasa si un hilo se bloquea para siempre esperando?

STM evita gran parte de todo eso por diseño.

Pretende replicar el comportamiento que tienen las transacciones en las bases de datos.

Ejemplo conceptual de STM

```
transaccion {  
  a = leer(x)  
  b = leer(y)  
  
  escribir(x, a + 1)  
  escribir(y, b + 1)  
}  
  
commit {  
  si versiones(x,y) no han cambiado desde que los leí:  
    aplicar cambios  
  si han cambiado:  
    abortar y reintentar  
}
```

¿Qué es una transacción?

Una transacción es una secuencia de operaciones sobre un conjunto de datos compartidos que puede terminar de 2 formas:

- Validando la transacción (commit).
- Abortando la transacción (roll-up).

Si solo una parte de la operación (ej. guardar información del administrador, pero no su *role*) se completa, es necesario un mecanismo para **deshacer** ese cambio parcial y restaurar la base de datos a un **estado consistente**. Ese mecanismo se llama **roll-back**.

Con esta técnica, las operaciones se ejecutan dentro de una transacción. De este modo, podemos disponer de operaciones libres de bloqueos e interbloqueos, ya que una transacción siempre termina de forma positiva o aborta la operación.

¿Cómo garantiza la exclusión mutua?

STM **no utiliza cerrojos** para proteger zonas críticas. Por lo que no garantizan la exclusión mutua en el sentido clásico de la palabra como hemos venido entendiéndolo hasta ahora.

Pero sí garantiza:

- **Atomicidad lógica** (los efectos de la transacción se ven como un todo indivisible).
- **Consistencia** (si algo falla, se deshacen los cambios).
- **Aislamiento** (las transacciones no ven modificaciones parciales de otras).

Sin embargo:

STM no garantiza ausencia absoluta de problemas de vivacidad.

El sistema puede entrar en situaciones de **livelock** si todas las transacciones chocan continuamente.

Pero algo importante:

✓ **En STM no puede haber interbloqueo clásico (deadlock)**

Porque no hay cerrojos que puedan quedar retenidos.

! **Sí puede haber livelock**

Todos los hilos avanzan pero ninguna transacción llega a commit porque se están abortando mutuamente.

¿Qué evita STM y qué no?

✓ **STM elimina el deadlock clásico**

No puedes quedarte esperando un cerrojo que nunca se libera, porque... ¡no hay cerrojos!

! **STM no evita completamente:**

- inanición (puedes abortar constantemente),
- livelock,
- starvation entre transacciones grandes vs pequeñas.

✓ STM simplifica la programación

No tienes que pensar en:

- protocolos de entrada/salida,
- quién desbloquea qué,
- en qué orden adquirir los cerrojos,
- si otro hilo me va a bloquear para siempre.

Clojure

Clojure es un lenguaje funcional sobre la JVM cuyo modelo de estado inmutable encaja perfectamente con STM. Tiene un manejador de transacciones.

Los valores son inmutables, y las identidades solo pueden cambiar dentro de una transacción.

Clojure dispone de:

- `Ref` → referencia mutable gestionada por STM
- `LockingTransaction.runInTransaction` → ejecuta código dentro de una transacción STM

```
import clojure.lang.Ref;
import clojure.lang.LockingTransaction;

Ref x = new Ref(0);
Ref y = new Ref(0);

LockingTransaction.runInTransaction(() → {
    x.set((int)x.deref() + 1);
    y.set((int)y.deref() + 1);
    return null;
});
```